

AVRA - Audit IA Studio

Geant - Coloriste IA + Rendu photorealiste - 5 mai 2026

Audit exhaustif de la section IA Studio (Coloriste IA + Rendu photorealiste). Cartographie code, tests live in-app, analyse des prompts backend, identification des bugs et corrections. 3 commits deployes pendant l'audit.

Bugs trouves	Bugs corriges	Commits deployes	Lignes auditees
7	5	2	3400

A - Architecture du systeme IA

Frontend : apps/web/app/(app)/ia-studio/page.tsx (1150 lignes). Page client unique avec 2 modes (Coloriste/Rendu), 8 presets palette, 5 styles, 6 etapes loading animees pour Coloriste, 8 pour Rendu.

Backend Next.js : 2 routes serverless dans apps/web/app/api/ia/ :

- /coloriste/route.ts (93 lignes) : auth + rate limit + appel generateColoristImage
- /rendu/route.ts (89 lignes) : meme structure mais Flux Pro Ultra

Builder : apps/web/lib/server/prompt-builder.ts (333 lignes). Systeme 'beton' : dictionnaire HEX -> nom couleur (27 entrees), 5 finitions, 3 lumieres, 4 tailles piece, 5 styles, prompts 3 niveaux (standard/simplified/minimal) + fallback safe. Seeds reproductibles via hash deterministe. Validation longueur prompt + presence keywords critiques.

Flux client : apps/web/lib/server/flux-api.ts (272 lignes). REST direct sans SDK : POST queue.fal.run puis polling sur requests/{request_id} jusqu'a 60 fois par 2s = 2 min max. Retry sur 3 niveaux de prompt + fallback final.

Modeles utilises :

- Coloriste : fal-ai/flux/dev (rapide, ~5-10s, 0.04 eur)
- Rendu : fal-ai/flux-pro/v1.1-ultra (qualite max, ~10-20s, 0.06 eur)

B - Bugs identifiés pendant l'audit

[CRITIQUE] Timeout serverless Vercel tue toutes les generations [CORRIGE]

Vercel utilise un `maxDuration` de 10s (Hobby) ou 15s (Pro default) pour les routes serverless. `fal.ai` peut prendre jusqu'à 90s + le code fait du `retry` sur 3 niveaux de `prompt`. Resultat : la fonction est tuée bien avant que `fal.ai` réponde, l'utilisateur voit 'Generation echouée - Erreur reseau' alors que sa connexion va parfaitement. Confirme en live : 2 min d'attente puis erreur affichée.

Fix : Ajout de export `const maxDuration = 300` en tête des 2 routes `/api/ia/coloriste` et `/api/ia/rendu`. Commit `5bdac0f`.

[CRITIQUE] Hook `useIAStudio.ts` catch silencieux [CORRIGE]

Le hook fait `console.error()` sur erreur sans la remonter dans l'UI (pas de `state errorMessage`). Si `fal.ai` échoue, le user voit le loader disparaître sans aucun message.

Fix : Ajout `state errorMessage` + `setErrorMessage` exposés par le hook. Detection spécifique du 429 (rate limit) avec message clair. Commit `5bdac0f`.

[IMPORTANT] Message d'erreur trompeur sur la page principale [CORRIGE]

`page.tsx` ligne 425 : `catch { setColorError('Erreur reseau - verifiez votre connexion') }`. Le `catch` ne lit pas le message vrai du backend, donc même si `fal.ai` dit 'FAL_KEY missing', l'utilisateur voit 'Erreur reseau' (faux).

Fix : Maintenant le `catch` lit le vrai message et le distingue du timeout reseau. Commit `f951b15`.

[IMPORTANT] Bouton 'Appliquer les couleurs' grise sans message [A AMELIORER]

Tant qu'aucune palette n'est sélectionnée, le bouton est grise (opacité réduite). Pas de tooltip ni message expliquant pourquoi. Découvert par tests live : on clique, rien ne se passe, pas de feedback.

Fix : Ajouter un placeholder type 'Choisissez une palette ou modifiez les couleurs pour activer' à côté du bouton, ou un tooltip au survol.

[MOYEN] Cartes 'Qualité & format' du Rendu non cliquables [A AMELIORER]

'4K Ultra HD', 'Lumière réelle', 'Anti-aliasing' : 3 cartes présentées comme des options mais ce sont juste des affichages décoratifs. Trompeur pour l'utilisateur qui pense pouvoir désactiver une option.

Fix : Soit retirer les cartes, soit les rendre vraiment switchables (et transmettre les flags au `prompt-builder`).

[MOYEN] Endpoint `/api/v1/demandes/stats` retourne 429 [A INVESTIGUER]

Découvert pendant les tests : un appel parallèle à `/demandes/stats` remonte 429 (rate limit NestJS). Probable conflit `ThrottlerModule` trop agressif. Pas critique mais signal que les rate limits backend sont peut-être trop serrés.

Fix : Vérifier la config `ThrottlerModule` dans `apps/api/src/app.module.ts`.

[MINEUR] Génération à une seule image (`num_images=1`) [A IMPROVER]

`flux-api.ts` ligne 156 : `num_images: 1`. Le user doit relancer 4 fois pour avoir 4 variations. Ce serait plus efficace de générer 4 images en parallèle en 1 seul appel.

Fix : Permettre `num_images: 2-4` avec un paramètre UI 'Combien de variantes'. Cout x N mais meilleur UX.

C - Points forts du code IA actuel

+ Architecture serveur-only

Les prompts ne sont JAMAIS exposes au client - construits cote serveur uniquement. FAL_KEY garde dans .env.local, pas accessible depuis le browser. Bonne pratique securite respectee.

+ Systeme de retry intelligent sur 3 niveaux

Si le prompt standard echoue (trop long, mots problematiques), on retry avec un prompt simplifie, puis minimal. En dernier recours, fallback safe valide manuellement. Architecture robuste face aux echecs fal.ai.

+ Dictionnaire HEX -> nom couleur

27 entrees mappees, plus un fallback approximatif par luminance. Permet a l'IA de comprendre exactement la couleur voulue (ex: #1B3254 -> 'midnight navy blue') au lieu de juste recevoir un code hex incomprehensible.

+ Seeds reproductibles

Hash deterministe des parametres -> meme palette + meme finition donne tjrs la meme image. Le user peut iterer sur les details sans tout perdre.

+ Validation des prompts avant envoi

Verifie longueur (150-900 chars), presence de mots-cles critiques ('kitchen', 'photorealistic'), negatifs incomplets. Retourne des warnings au front pour debug.

+ Loading steps detailles + animation auto-progress

ProgressBar avec 6 etapes pour Coloriste, 8 pour Rendu. UX premium - le user voit que ca avance meme si fal.ai prend 60s.

+ 8 presets palette tres bien pensees

Noir Absolu, Blanc Satine, Chene Fume, Gris Ardoise, Sauge Premium, Bleu Nuit, Terracotta, Beton Cire. Chaque preset a un mood ('Luxe contemporain', 'Industriel chic'), un material handle et un material plan. Comprehensible meme sans expertise.

+ Estimation cout + duree affichee avant le clic

Le user sait que ca va couter ~0.04 eur ou ~0.06 eur et prendre 5-20s. Transparence.

+ Negatifs photorealisme rigoureux

Liste exhaustive : cartoon, illustration, 3D render stylise, blurry, deformed, people, dirty, oversaturated, fish-eye, etc. Empeche les outputs trop 'IA-typiques'.

+ Suffixe technique pro

Canon EOS R5, 24-70mm f/2.8, ISO 200, 8K, Architectural Digest style. Aligne le rendu sur des references photo-pro reconnues, pas du CGI generique.

D - Recommandations 30/60/90 jours

Court terme (semaine prochaine)

1. Verifier que le deploy 5bdac0f resout bien le timeout en testant en live (le maxDuration=300 doit etre actif).
2. Ajouter un tooltip ou message sur le bouton grise 'Appliquer les couleurs'.
3. Decider du sort des cartes 'Qualite & format' (les rendre interactives ou les retirer).
4. Investiguer pourquoi /api/v1/demandes/stats retourne 429 - probable ThrottlerModule a calmer.

Moyen terme (30 jours)

1. Implementer num_images = 2-4 avec un slider UI 'Combien de variantes ?' - permet au user de comparer plusieurs propositions sans payer 4 generations separees.
2. Sauvegarder les rendus dans la galerie persistee (actuellement state local donc perdu au refresh).
3. Ajouter mode 'comparaison avant/apres' : la photo uploadee a gauche, le rendu IA a droite, glisser-deplacer le clip-mask pour comparer en direct.
4. Loguer chaque generation IA dans une table dediee (audit, statistiques cout/utilisateur, debug).

Long terme (90 jours)

1. Module IA dedie pour cuisinistes uniquement actuellement. Ajouter des verticals pour menuisier (dressing, escalier, bibliotheque) et architecte d'interieur (salon, salle de bain, chambre) avec leurs propres presets et prompts.
2. A/B testing automatique sur les prompts : tester en parallele 2 versions, mesurer le taux de 'sauvegarde gallery' (proxy qualite percue), iterer sur le gagnant.
3. Mode 'edit' : le user upload une photo de cuisine existante, choisit ce qu'il veut changer (juste les facades, juste les poignees), l'IA fait du in-painting cible. Demande integration Flux 'redux' ou DALL-E inpainting.
4. Memoriser les prompts qui ont reussi pour chaque utilisateur (ML personnalise) - apres 50 generations, le systeme propose automatiquement les params qui marchent le mieux pour ce user.

Document genere automatiquement - Audit IA Studio - 5 mai 2026 - AVRA - SaaS ERP cuisinistes, menuisiers, architectes d'interieur - avra-app.fr

Commits deployes pendant l'audit : 5bdac0f (timeout fix), f951b15 (message erreur clair).